

GUIA DE ESTUDO - JAVA BACKEND

Tratamento global de exceções

Como o Medflow transforma falhas de domínio, validação e infraestrutura em respostas HTTP consistentes e seguras.

STACK	CONTRATO	ESCOPO
Java 21 Spring Boot 4.1.1	Problem Details RFC 9457	Patient + Exam Validação + Segurança

Projeto Medflow

Material técnico para revisão de Vinícius Oliveira

SEÇÃO 01

O problema que estamos resolvendo

Uma API precisa falhar de modo previsível. O cliente não deve conhecer as classes internas da aplicação nem receber um formato diferente em cada endpoint.

Requisição → Controller → Exceção → Handler → ProblemDetail

Ideia central

A camada de negócio continua responsável por detectar a falha e lançar uma exceção. A camada web decide como essa falha será apresentada por HTTP. Essa divisão evita acoplamento entre regra de negócio e protocolo.

Sem tratamento global

Cada Controller repete try/catch, escolhe formatos diferentes e pode vaziar detalhes internos.

Com tratamento global

Um único ponto traduz exceções para status, mídia e estrutura estáveis.

Regra mental

Quem detecta não precisa formatar

Service lança a exceção.
GlobalExceptionHandler escolhe o status e monta a resposta. Controller mantém o fluxo normal.

SEÇÃO 02

Por que shared.web.error?

O nome do pacote comunica alcance, camada e responsabilidade. Não é uma arquitetura nova: é apenas uma casa clara para uma preocupação transversal da borda HTTP.

Parte	Significado	Decisão no Medflow
shared	Compartilhado entre módulos.	Patient e Exam usam o mesmo contrato de erro.
web	Pertence à interface HTTP.	Não coloca ProblemDetail dentro do domínio.
error	Agrupar a tradução de falhas.	Evita espalhar handlers pelos Controllers.

Os três arquivos

GlobalExceptionHandler

Orquestra a tradução: recebe exceção, escolhe status e cria ProblemDetail.

ProblemTypes

É o catálogo das URNs estáveis que identificam cada categoria de problema.

ValidationError

É o contrato de um erro de campo: mensagem legível e localização no JSON.

Quando dividir mais?

Somente quando o handler crescer por novos módulos. Hoje, três arquivos são suficientes.

Leitura profissional

- Separar arquivos não significa aplicar Clean Architecture completa.
- Cada arquivo possui uma responsabilidade e um motivo concreto para existir.
- Juntar tudo reduziria navegação, mas aumentaria mistura de conceitos dentro do handler.
- O nível técnico está no entendimento das decisões, não na quantidade de classes.

SEÇÃO 03

Problem Details e RFC 9457

ProblemDetail oferece uma linguagem comum para erros HTTP. Clientes podem tratar a categoria do problema sem depender de mensagens humanas instáveis.

Campo	Função	Exemplo
type	Identificador estável da categoria.	urn:medflow:problem:patient-not-found
title	Resumo curto e estável.	Paciente não encontrado
status	Código HTTP também presente no corpo.	404
detail	Explica aquela ocorrência.	Paciente com id ... não encontrado
instance	Identifica a requisição afetada.	/patients/{id}
errors	Extensão para violações de campos.	detail + pointer

Exemplo de resposta

```
{
  "type": "urn:medflow:problem:patient-not-found",
  "title": "Paciente não encontrado",
  "status": 404,
  "detail": "Paciente com id: 76cb... não encontrado",
  "instance": "/patients/76cb..."
}
```

Por que uma URN?

O campo type precisa de um identificador URI estável. A URN cumpre esse papel sem exigir que o projeto possua um domínio público.

SEÇÃO 04

Escolha dos status HTTP

O status deve descrever o tipo de falha observado pelo cliente. O corpo não substitui o status: ambos precisam contar a mesma história.

Status	Quando usar	Cenário no Medflow
400	A requisição não pôde ser interpretada corretamente.	JSON quebrado ou UUID em formato inválido.
404	O recurso solicitado não existe.	Paciente ou ordem de exame não encontrados.
422	A sintaxe foi entendida, mas os dados violam o contrato.	CPF inválido, nome vazio ou data futura.
500	O servidor encontrou uma falha inesperada.	Exceção sem tratamento mais específico.

Como o Spring entra no fluxo

- `@RestControllerAdvice` torna o handler visível para todos os Controllers.
- `@ExceptionHandler` liga uma exceção de domínio a um método específico.
- `ResponseEntityExceptionHandler` oferece pontos de extensão para falhas geradas pelo próprio Spring MVC.
- Os overrides preservam os headers do Spring e substituem apenas o corpo e o status necessários.

Por que não colocar try/catch no Controller?

Porque o Controller deveria coordenar entrada e saída. Repetir tratamento em cada endpoint aumenta duplicação e facilita contratos inconsistentes.

SEÇÃO 05

Validação: do DTO ao campo inválido

Quando `@Valid` encontra violações no `PatientDtoRequest`, o Spring lança `MethodArgumentNotValidException` antes de executar o método do Controller.

Transformação realizada pela Stream

```
List<ValidationError> errors = exception.getBindingResult()
    .getFieldErrors()
    .stream()
    .map(fieldError -> new ValidationError(
        fieldError.getDefaultMessage() != null
        ? fieldError.getDefaultMessage()
        : "Valor inválido",
        toJsonPointer(fieldError.getField()))
    )
    .sorted((first, second) ->
        first.pointer().compareTo(second.pointer()))
    .toList();
```

Etapas	Responsabilidade
getFieldErrors	Obtém somente as violações associadas a campos.
stream	Inicia o processamento declarativo da coleção.
map	Transforma <code>FieldError</code> do Spring em <code>ValidationError</code> da API.
sorted	Garante ordem determinística pelo JSON Pointer.
toList	Materializa o resultado que será colocado em <code>errors</code> .

JSON Pointer

O pointer localiza o campo no documento recebido. Um caminho Java como `address.street` vira `#/address/street`. Os caracteres reservados `~` e `/` são escapados conforme a RFC 6901.

SEÇÃO 06

Segurança e falhas inesperadas

Uma exceção inesperada precisa gerar duas saídas diferentes: informação completa para quem opera o servidor e informação segura para o consumidor da API.

Log do servidor

Recebe mensagem, classe da exceção e stack trace. Serve para investigação técnica.

Resposta do cliente

Recebe um ProblemDetail estável, sem SQL, stack trace, credenciais ou detalhes internos.

```
@ExceptionHandler(Exception.class)
public ResponseEntity<ProblemDetail> handleUnexpected(
    Exception exception,
    HttpServletRequest request) {

    LOGGER.error("Erro não tratado ao processar {}",
        request.getRequestURI(), exception);

    return createResponse(
        HttpStatus.INTERNAL_SERVER_ERROR,
        ProblemTypes.INTERNAL_ERROR,
        "Erro interno",
        "Ocorreu um erro interno inesperado.",
        URI.create(request.getRequestURI())
    );
}
```

Regra de segurança

Nunca use `exception.getMessage()` no corpo de um erro 500 genérico. A mensagem pode conter detalhes de banco, caminhos, tokens ou dados sensíveis.

SEÇÃO 07

Como adicionar um novo tratamento

O processo deve começar pela semântica da falha, não pelo status HTTP. Primeiro modele a exceção; depois defina como a borda web a representa.

Passo	Pergunta de revisão
1. Exceção	Ela representa uma falha real do domínio ou da aplicação?
2. Type	A URN é única, estável e adicionada em ProblemTypes?
3. Status	O código HTTP descreve corretamente o que o cliente pode fazer?
4. Detail	A mensagem ajuda sem revelar informação interna?
5. Teste	Status, content-type, campos RFC e instance foram verificados?

Exemplo

```
@ExceptionHandler(DoctorNotFoundException.class)
public ResponseEntity<ProblemDetail> handleDoctorNotFound(
    DoctorNotFoundException exception,
    HttpServletRequest request) {
    return createResponse(
        HttpStatus.NOT_FOUND,
        ProblemTypes.DOCTOR_NOT_FOUND,
        "Médico não encontrado",
        exception.getMessage(),
        URI.create(request.getRequestURI())
    );
}
```

O que saber explicar em uma entrevista

- A regra de negócio lança a exceção; a camada web traduz para HTTP.
- ProblemDetail cria um contrato previsível e interoperável.
- 400, 404, 422 e 500 representam falhas diferentes.
- O catch-all protege o cliente e mantém o diagnóstico no log.

SEÇÃO 08

Testes e checklist final

Os testes desta entrega exercitam o handler e o Validator reais por meio do MockMvc, sem Mockito e sem substituir Service ou Repository por mocks.

Cenário	Evidência protegida
Paciente ausente	404 + type patient-not-found + instance correto.
Ordem ausente	404 + type exam-order-not-found.
DTO inválido	422 + lista errors ordenada com JSON Pointer.
JSON quebrado	400 + mensagem pública estável.
UUID inválido	400 antes da execução do Controller.
Falha inesperada	500 sem vazamento da mensagem interna.

Comando de verificação

```
.\mvnw.cmd test

Tests run: 44, Failures: 0, Errors: 0
BUILD SUCCESS
```

MockMvc não é Mockito

MockMvc simula a infraestrutura HTTP sem abrir uma porta de servidor. Nesta entrega, GlobalExceptionHandler e Validator são objetos reais; não há mocks de interação entre classes.

Referências

RFC 9457 - Problem Details for HTTP APIs
<https://www.rfc-editor.org/rfc/rfc9457.html>

RFC 9110 - HTTP Semantics
<https://www.rfc-editor.org/rfc/rfc9110.html>

RFC 6901 - JavaScript Object Notation Pointer
<https://www.rfc-editor.org/rfc/rfc6901.html>

Spring MVC - Error Responses
<https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-ann-rest-exceptions.html>